P2_L3

Enunciado

El equipo de desarrollo no tiene conocimiento de sistemas y debe hacer que el servidor tenga configurado un servidor web para poder ejecutar Wordpress así que decide instalar y configurar la pila (el stack) LAMP.

Instalamos toda la pila LAMP con el siguiente comando: " dnf install httpd php mariadb mariadb-server php-mysqlnd"

http -> Apache HTTP Server. Es el servidor web que atiende las peticiones HTTP de los navegadores.

Lo primero que vamos a hacer con http es comprobar el estado del servicio con el comando "systemctl status httpd", y vemos que está inactivo.

```
[root@localhost marcosgal]# systemctl status httpd
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
    Drop-In: /etc/systemd/system/httpd.service.d
             └─php-fpm.conf
     Active: inactive (dead)
       Docs: man:httpd.service(8)
```

Para habilitarlo cuando iniciemos el sistema ejecutamos el comando "systemctl enable httpd", y para arrancarlo en el momento usamos "systemctl start httpd", y comprobamos.

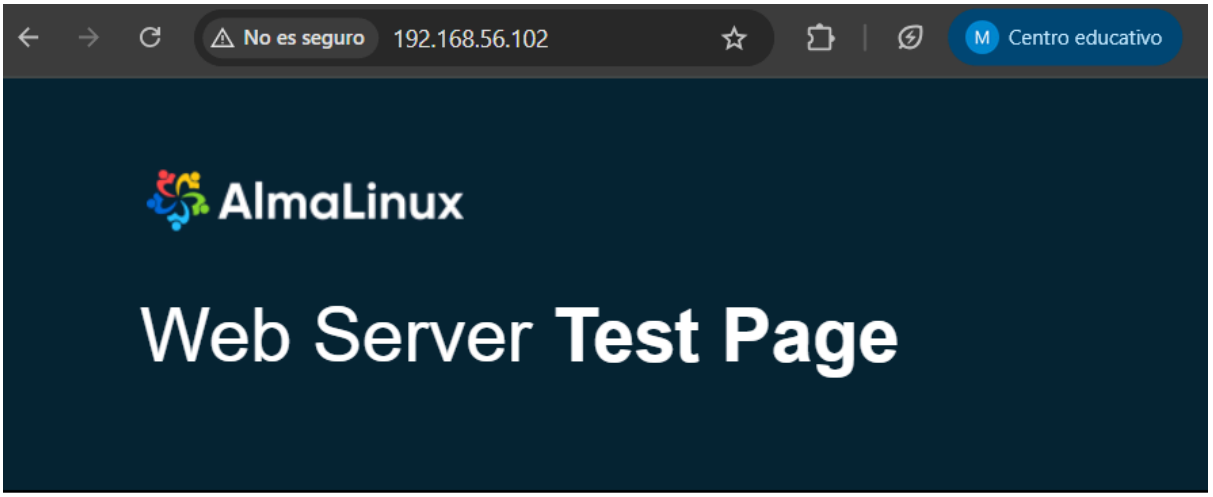
```
[root@localhost marcosgal]# systemctl start httpd
[root@localhost marcosgal]# systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
    Drop-In: /etc/systemd/system/httpd.service.d
             └─php-fpm.conf
     Active: active (running) since Sat 2025-10-18 12:28:25 CEST; 1s ago
 Invocation: 5028fd9134ac4ab79983793f40921d06
       Docs: man:httpd.service(8)
   Main PID: 1414 (httpd)
     Status: "Started, listening on: port 80"
      Tasks: 177 (limit: 10699)
     Memory: 13.3M (peak: 13.6M)
        CPU: 69ms
     CGroup: /system.slice/httpd.service
             ├─1414 /usr/sbin/httpd -DFOREGROUND
             ├─1423 /usr/sbin/httpd -DFOREGROUND
             ├─1424 /usr/sbin/httpd -DFOREGROUND
             ├─1425 /usr/sbin/httpd -DFOREGROUND
             └─1426 /usr/sbin/httpd -DFOREGROUND
```

Otra forma de comprobar que Apache está funcionando es con el comando "curl 127.0.0.1", este es una herramienta que nos permite hacer solicitudes a un

servidor y mostrar la respuesta en pantalla. Vemos que funciona correctamente.

```
                </div>
        <div class="footer">
                <a href="https://apache.org">A
he United States and/or other countries.
        <br />
                <a href="https://nginx.com">NG
        </div>
        </body>
</html>
```

Al intentar conectarnos desde nuestra máquina anfitrión poniendo http://(ip de la VM) vemos que la página falla mostrándonos que no ha podido conectarse. Esto ocurre porque el firewall está bloqueando el puerto 80 que es el que utiliza Apache. Para arreglar esto es nuestra VM ejecutamos los siguientes comandos: "firewall-cmd --add-port=80/tcp --permanent" y "firewall-cmd --reload" para que el firewall deje de bloquear el puerto 80, ahora volvemos a comprobar en nuestra maquina anfitrion si funciona y vemos que si.



php -> Es un intérprete de lenguaje que funciona integrado dentro del servidor web Apache

Comprobamos que php está instalado con "php -a"

```
[root@localhost marcosga]# php -a
Interactive shell
```

mariabd-server-> Es una base de datos

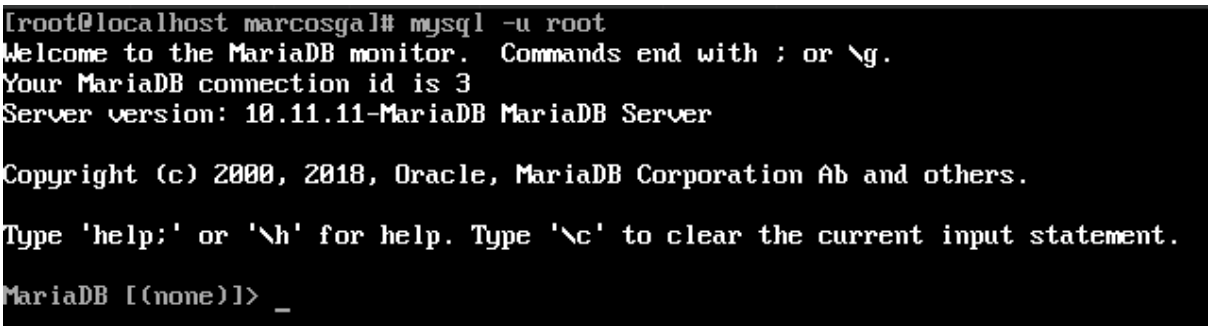
Ejecutamos “systemctl status mariadb” y vemos que esta inactivo



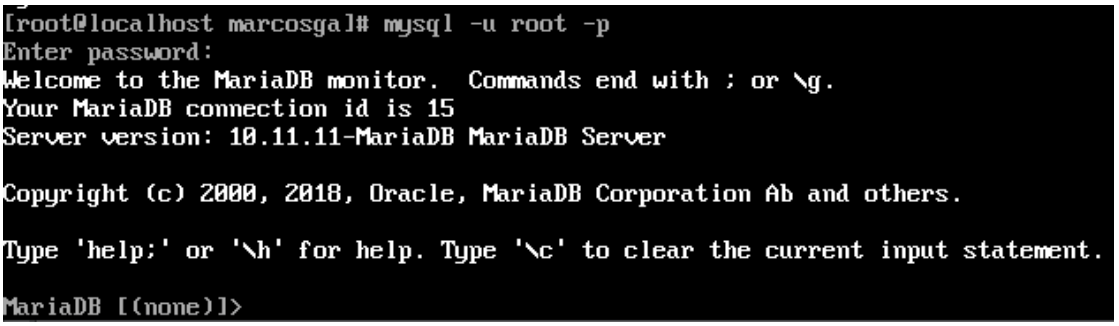
Para activar el servicio usamos “systemctl enable mariadb” para que cada vez que arranquemos el sistema se inicie y “systemctl start mariadb” para activarlo en el momento.



Entramos al gestor de la base de datos desde el terminal con el siguiente comando: “mysql -u root” .



Para proteger la base de datos ejecutamos “mysql_secure_installation” para añadir una contraseña, en este caso “practicas,ise”. Y ahora al ejecutar “mysql -u root -p” vemos que nos pide contraseña para acceder a la base de datos.



Una vez hemos comprobado que todos los componentes funcionan correctamente por separado vamos a crear una página index.php para ver que todo funciona.

Creamos nuestra página en php en /var/www/html/index.php. Al conectarnos luego a la página nos debe salir un mensaje llamado "Connected successfully" si todo ha funcionado correctamente, si no nos debe salir "Connection failed".

```
<?php
$servername = "localhost";
$username = "root";
$password = "practicas,ise";

try {
  $conn = new PDO("mysql:host=$servername;dbname=myDB", $username, $password);
  $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
  echo "Connected successfully";
} catch(PDOException $e) {
  echo "Connection failed: " . $e->getMessage();
}
?>
```

Ahora accedemos a la base de datos y añadimos myDB con el comando "CREATE DATABASE myDB".

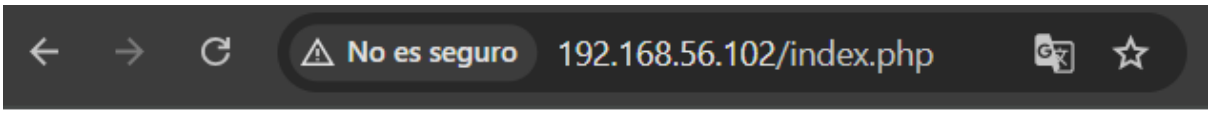
```
[root@localhost marcosga]# mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 16
Server version: 10.11.11-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE myDB;
Query OK, 1 row affected (0.001 sec)
```

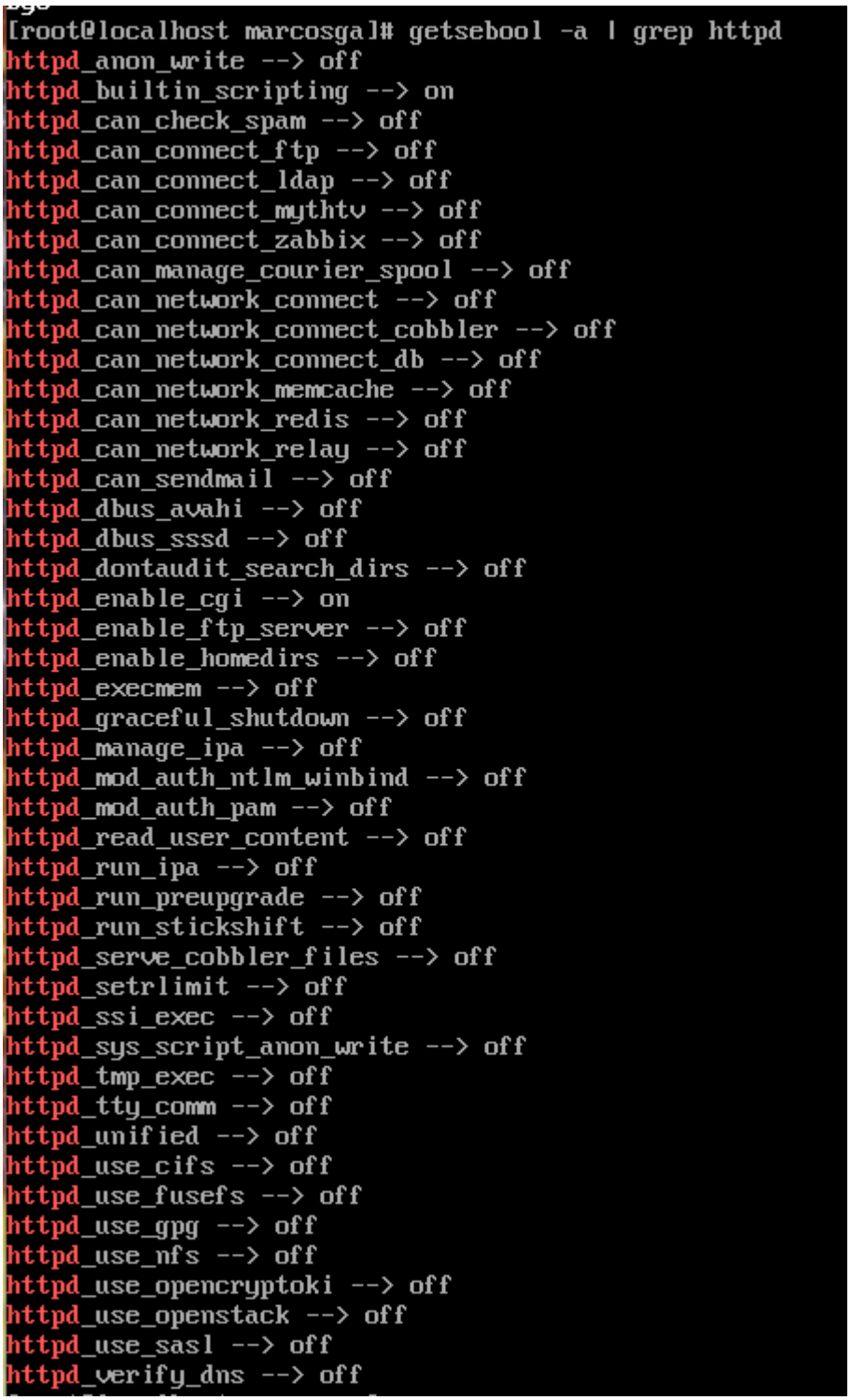
Y accedemos al sitio web desde un navegador anfitrión.



Connected successfully

Vemos que todo funcionó correctamente.

Comprobamos todas las variables booleanas con el comando "getsebool -a | grep httpd" para ver la lista de las políticas selinux relacionadas con httpd.



Vemos que algunas están activadas y otras no, activamos alguna como httpd_can_network_connect_db que permite que Apache se conecte a una base de datos a través de una red.

```
[root@localhost marcosga]# setsebool -P httpd_can_network_connect_db=on
[ 3614.632140] SELinux:  Converting 351 SID table entries...
[ 3614.637145] SELinux:  policy capability network_peer_controls=1
[ 3614.637264] SELinux:  policy capability open_perms=1
[ 3614.637361] SELinux:  policy capability extended_socket_class=1
[ 3614.637449] SELinux:  policy capability always_check_network=0
[ 3614.637537] SELinux:  policy capability cgroup_seclabel=1
[ 3614.637653] SELinux:  policy capability nnp_nosuid_transition=1
[ 3614.637759] SELinux:  policy capability genfs_seclabel_symlinks=1
[ 3614.637889] SELinux:  policy capability ioctl_skip_cloexec=0
[ 3614.637995] SELinux:  policy capability userspace_initial_context=0
```

```
httpd_can_network_connect_db --> on
```

Comprobamos que se activó la política correctamente y finalizamos la lección.